southworks

Golf Ball Tracking

Real-Time Detection on Broadcast Video

Technical Overview v0.1



Summary

A working prototype that finds and follows the golf ball in broadcast video in real time, and a proposed AWS architecture to run it on recorded and live feeds.

Highlights

- **Real time on a laptop:** YOLOE-26s and a custom tracker process the full broadcast test video at 29.5 fps, dropping 1.5% of frames, with 30 ms median latency.
- **Keeps the ball:** on recorded video the ball is boxed on 96.5% of the frames where it is visible (1,215 hand-labelled frames). Misses fell 63% against the starting point.
- **Built for broadcast:** scene cuts, whip-pans that follow the ball, motion blur and burned-in graphics are handled, and the annotated video can be re-streamed live.
- **Main open issue:** boxes on look-alikes, mostly tee markers, now outnumber missed balls about 7 to 1. Fine-tuning the detector is the fix.
- **On AWS:** GPU batch jobs (AWS Batch) for recorded video; for live, GPU nodes on Amazon EKS with SRT in and out through AWS Elemental MediaConnect.
- **Not measured yet:** speed on AWS NVIDIA GPU instances, glass-to-glass latency on a real feed, 50/60 fps input.

Current Implementation

What It Looks Like (1/3)



432.6 s · ball descending against the sky



433.3 s · the camera whip-pans after it



433.7 s · still boxed as it lands by the flag



Debug view: hidden box, expected path, search crop



Putt: the trail stays on the green as the camera moves

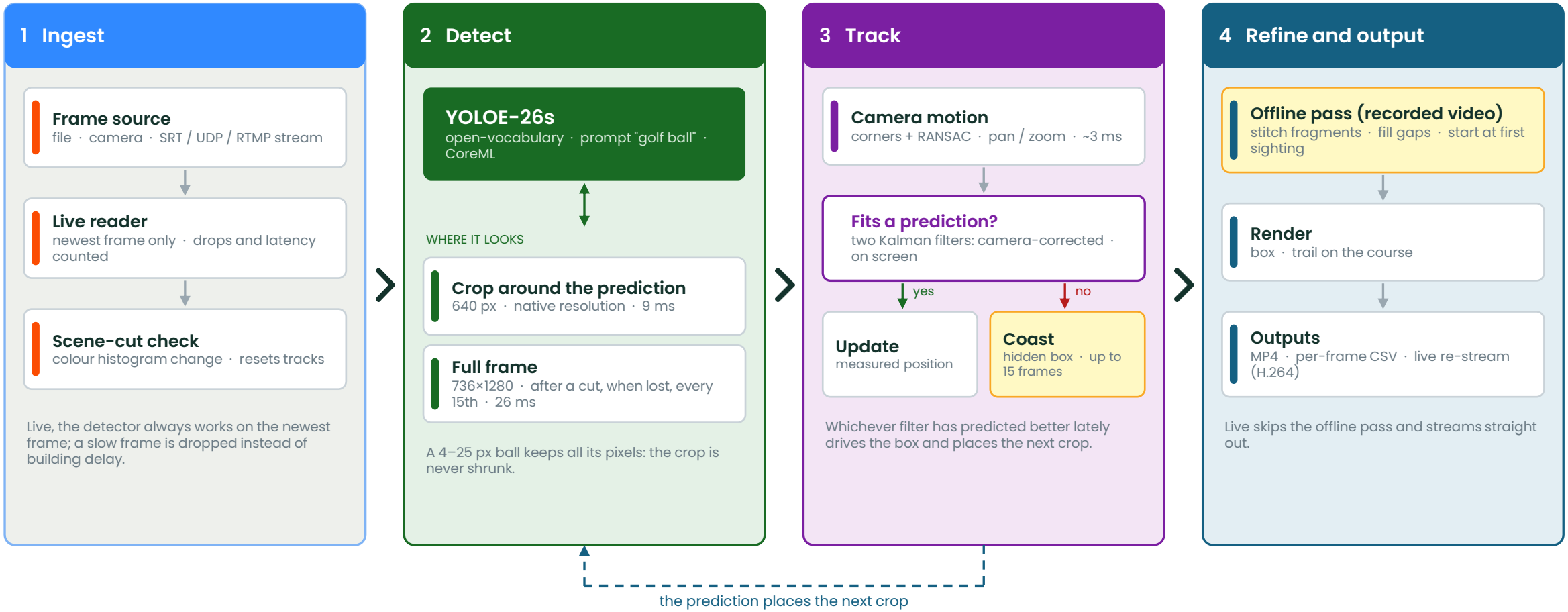
How to read it

- **Green box** the ball is detected on this frame
- **Orange box** filled in between two real sightings
- **Yellow trail** the ball's path on the course, not the screen
- **Cyan dots** debug only: where the tracker expects the ball next

Current Implementation

How the Pipeline Works (2/3)

One frame through the tracker



Current Implementation

Technical Approach (3/3)

Each choice below came from a measurement on broadcast footage, recorded with its evidence in docs/decisions.md. The whole prototype runs on a single laptop.

Aspect	Decision	Rationale
Detector	YOLOE-26s, an open-vocabulary YOLO, prompted with "golf ball"	Best of 12 configurations on our benchmark (F1 0.79). Needs no golf training data.
Where it looks	640 px crop around the predicted ball at native resolution; a full frame after cuts, when lost, and every 15th frame	About 2× faster than a full frame and finds as many or more balls, because a 4–25 px ball is never downscaled
Runtime	Optimised on-device runtime (CoreML), FP16, one model per input size	Crop 29 → 9 ms, full frame 62 → 26 ms. On AWS the equivalent is TensorRT or ONNX Runtime on NVIDIA GPUs.
Tracker	Own Kalman tracker with two motion hypotheses per track: camera-corrected and on screen	Box-overlap trackers (ByteTrack, BoT-SORT) fail when the ball moves more than its own size per frame; one model cannot follow both a still and a whip-panning camera
Camera motion	Corner tracking and RANSAC give pan and zoom each frame (~3 ms)	Keeps predictions and the trail on the course, which a tracer needs
Scene cuts	Colour-histogram change between frames resets the tracks	Highlights cut every few seconds; a track must never jump across shots
Recorded video	Offline pass: stitch track fragments of the same ball, fill gaps between sightings	Uses the future: losses 45 → 24 and ID changes 49 → 9 on top of the two-hypothesis tracker
Live input and output	Reader that always takes the newest frame; re-stream through ffmpeg with hardware H.264	A slow frame is dropped instead of adding delay; tested end to end over UDP

Results

Results on Broadcast Test Footage

29.5 fps

processed live on a standard laptop, full 8.5 min test video

30 ms

median latency, frame in to annotated frame out

96.5%

of frames with a visible ball have a box on it (recorded)

63%

fewer missed balls than at the starting point

Keeping the ball, checked against hand labels (1,215 frames with a visible ball, two videos). Each row adds one change.

Tracker version	Box on ball	Missed	Wrong object	Box with no ball	Recall
Starting point: one Kalman filter	1,132	60	23	188	93.2%
Search follows the prediction	1,123	64	28	188	92.4%
Two motion hypotheses (works live)	1,146	45	24	206	94.3%
Offline pass (recorded video)	**1,173**	**22**	**20**	**162**	**96.5%**

Live: the starting setup (YOLOE-26n, full frame) ran at 25.0 fps and dropped 16.5% of frames; YOLOE-26s with an optimised runtime and the crop runs at 29.5 fps and drops 1.5%.

Next problem: boxes drawn when no ball is in the frame (162) now outnumber misses (22), mostly on tee markers.

Considerations

What Makes Broadcast Golf Hard

Consideration	What it means	How it is handled
Broadcast editing	Hard cuts every few seconds; a track must not jump across shots	Cut detector resets tracks; caught all 40 cuts on the test video
Camera follows the ball	Whip-pans of 100–240 px per frame while the ball barely moves on screen	Camera-motion estimate plus two motion hypotheses per track
Tiny, blurred ball	4–25 px at 1080p, a streak in flight at 30 fps	Native-resolution crops, 1280 px full frames, prediction across short gaps
Look-alikes	Tee markers, cups, club heads, the broadcast's own tracer graphics	**Open:** fine-tune with them as negatives (162 false-box frames against 22 misses)
No labelled dataset	Public golf models are trained on phone video, not broadcast	Hand-labelled test sets: 1,299 detector and 1,614 tracking judgements
Runtime portability	The prototype's optimised runtime is laptop-specific; AWS GPU instances are NVIDIA	Export to TensorRT or ONNX Runtime and re-benchmark on AWS (G5 / G6)
Licensing	Ultralytics YOLO is AGPL-3.0; broadcast footage has usage rights	Enterprise licence or AGPL-compliant release; confirm rights before training on broadcasts
Recorded vs live	The offline pass gives the best result but needs the future	Live gets everything else; a short look-ahead buffer could close the gap

Improvements

Possible Improvements

Fine-tune the detector

Train YOLOE-26s with tee markers, cups, club heads and tracer graphics as negatives. The hand labels are the test set.

Impact: High | Effort: 2–3 days | Recorded + live

Broadcast-style tracer

Fit a flight curve per shot on the stitched track, in scene coordinates, and draw it instead of the raw trail.

Impact: High | Effort: Medium | Recorded + live

Look-ahead buffer

Hold 0.5–1 s of video so live can stitch fragments and fill gaps like the offline pass.

Impact: High | Effort: Medium | Live

Render every frame

Draw skipped frames from the prediction, so the re-stream never repeats a frame.

Impact: Medium | Effort: Small | Live

Recover the last gaps

Search for the best path through weak detections that no track used.

Impact: Medium | Effort: Medium | Recorded

Ball data and stats

Per-frame position as a feed for graphics engines; shot stats such as hang time.

Impact: Medium | Effort: Medium | Live

Next Steps

Next Steps



1 Fine-tune the detector

Hard negatives and more broadcast footage. Target: far fewer false boxes, recall at or above 96.5%.

2 AWS recorded-video proof of concept

Containerise, TensorRT on an NVIDIA GPU instance, AWS Batch. Re-benchmark speed and accuracy.

3 Live proof of concept

EKS GPU nodes with SRT in and out through MediaConnect. Measure glass-to-glass latency on a test feed.

4 Tracer and look-ahead

Broadcast-style tracer; a 0.5–1 s live buffer to bring offline quality to live.

5 Real feed trial

One event with the broadcaster's contribution feed, monitored end to end.

Decisions needed to start

- **Training data:** fine-tune on our two test videos only (faster) or download more broadcast footage (more robust).
- **AWS:** account and region, and G5 / G6 GPU instance quota for the proofs of concept.
- **Licensing:** Ultralytics Enterprise licence or an AGPL-compliant deployment.
- **Target feed:** protocol, resolution, frame rate, and how much delay the annotated feed may add.

Discussion Items

Questions

1. **Feed:** which protocol (SRT, RTMP, SDI through a gateway), resolution and frame rate (1080p50/60, UHD)?
2. **Latency:** how much delay may the annotated feed add? Is a 0.5–1 s look-ahead acceptable for better tracking?
3. **Output:** a burned-in overlay, a data feed for the broadcaster's graphics engine, or both?
4. **Content rights:** can broadcast footage be used to train and evaluate the model?
5. **Licensing:** Ultralytics YOLO is AGPL-3.0. Enterprise licence, or keep the deployment AGPL-compliant?
6. **AWS:** which account and region, and can we request GPU instance quota now?
7. **Scope:** golf only, or other small-ball sports later?

southworks

More Details

Technical details and proposed architecture



Approach

Measure First, Then Change (1/2)

There was no labelled golf dataset for broadcast video, so we built the measurements before the features. Every change was kept only if the numbers moved.

1 Benchmark the detector

12 detector configurations on 236 broadcast frames, scored against 1,299 pooled hand judgements.

Result: YOLOE-26s chosen, F1 0.79

`tools/eval_detectors.py`

2 Simulate live

Recorded broadcasts replayed in real time and in a loop. Frames the detector cannot keep up with are dropped and counted, as a real feed would.

Result: crop and optimised runtime: 29.5 fps, 1.5% dropped

`golf-track --live --loop`

3 Diagnose losses

Detections recorded once, then the tracker replayed in seconds. The worst losses inspected frame by frame: whip-pans, predictions coasting off screen.

Result: two motion hypotheses, offline pass

`tools/eval_tracking.py`

4 Verify with ground truth

1,614 candidates hand-labelled on every 10th frame of two videos. Each output scored as on the ball, missed, wrong object or false box.

Result: recall 93.2% to 96.5%

`tools/eval_track_gt.py`

Every change has a before/after video on the same moments and a row in docs/decisions.md.

Approach

Choosing the Detector (2/2)

A golf ball is 4–25 px in a 1080p broadcast. We compared general, open-vocabulary and golf-specific detectors on the same frames, at 1280 px input, which roughly doubles recall over 640 px.

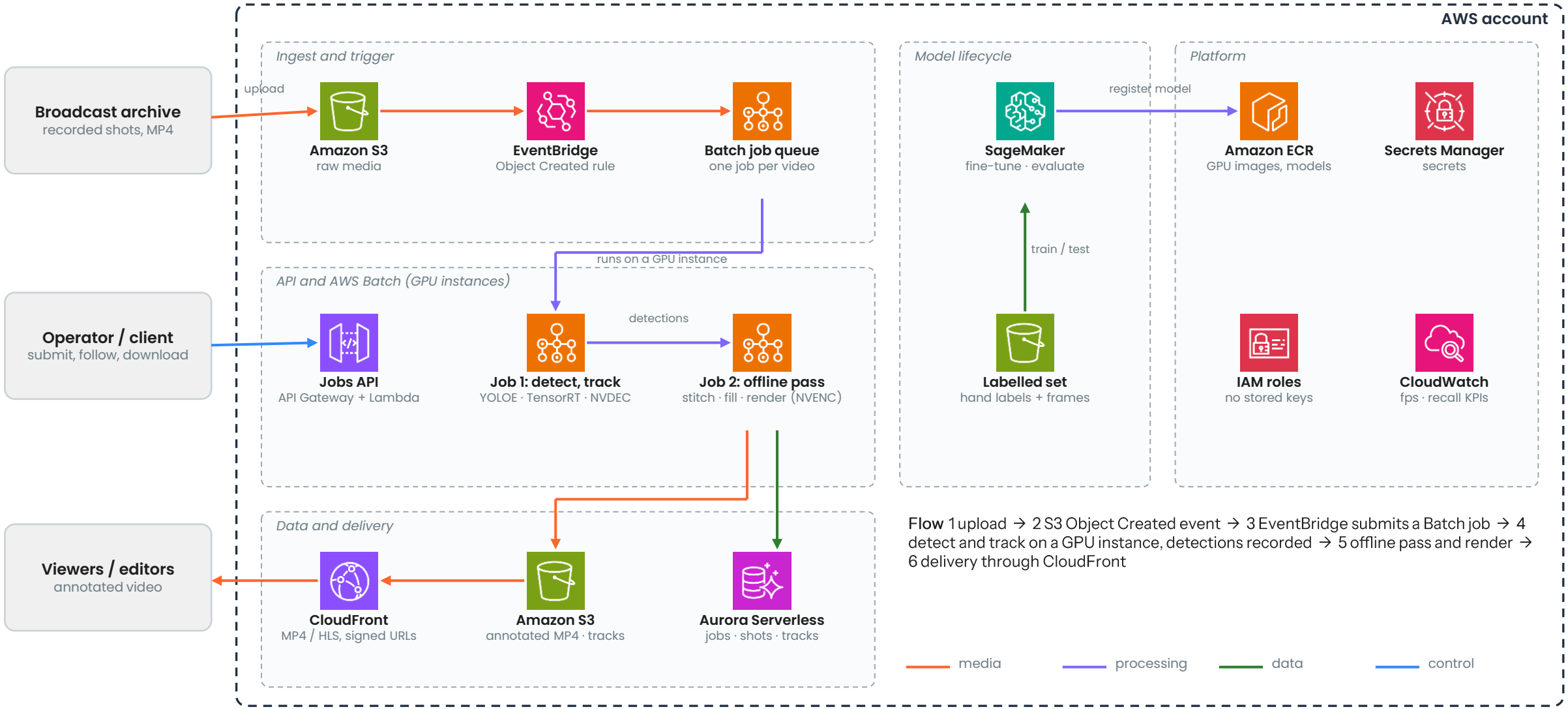
Configuration	Type	Best F1	Speed (laptop)	Verdict
YOLOE-26s "golf ball"	Open-vocabulary	0.79	16 fps	Chosen: most accurate; live once cropped and optimised
YOLOE-26n "golf ball"	Open-vocabulary	0.74	31 fps	Fast fallback
YOLO26l	COCO "sports ball"	0.59	8 fps	Slower and less precise
YOLO-World v2 s "golf ball"	Open-vocabulary	0.54	22 fps	Weaker on small balls
YOLO26n	COCO "sports ball"	0.54	48 fps	Fast, many false positives
Golf YOLOv8n (Hugging Face)	Trained on phone video	0.19	28 fps	Rejected: wrong domain

Projects reviewed

- **Two GitHub golf trackers and one Hugging Face model:** none reusable (a README only, a phone-video model, a tile-trained model with no tracker or licence).
- **Ideas kept:** detect on a crop around the ball, compensate camera motion, hold out whole clips when fine-tuning.

Architecture

Recorded Video (VOD) on AWS (1/2)



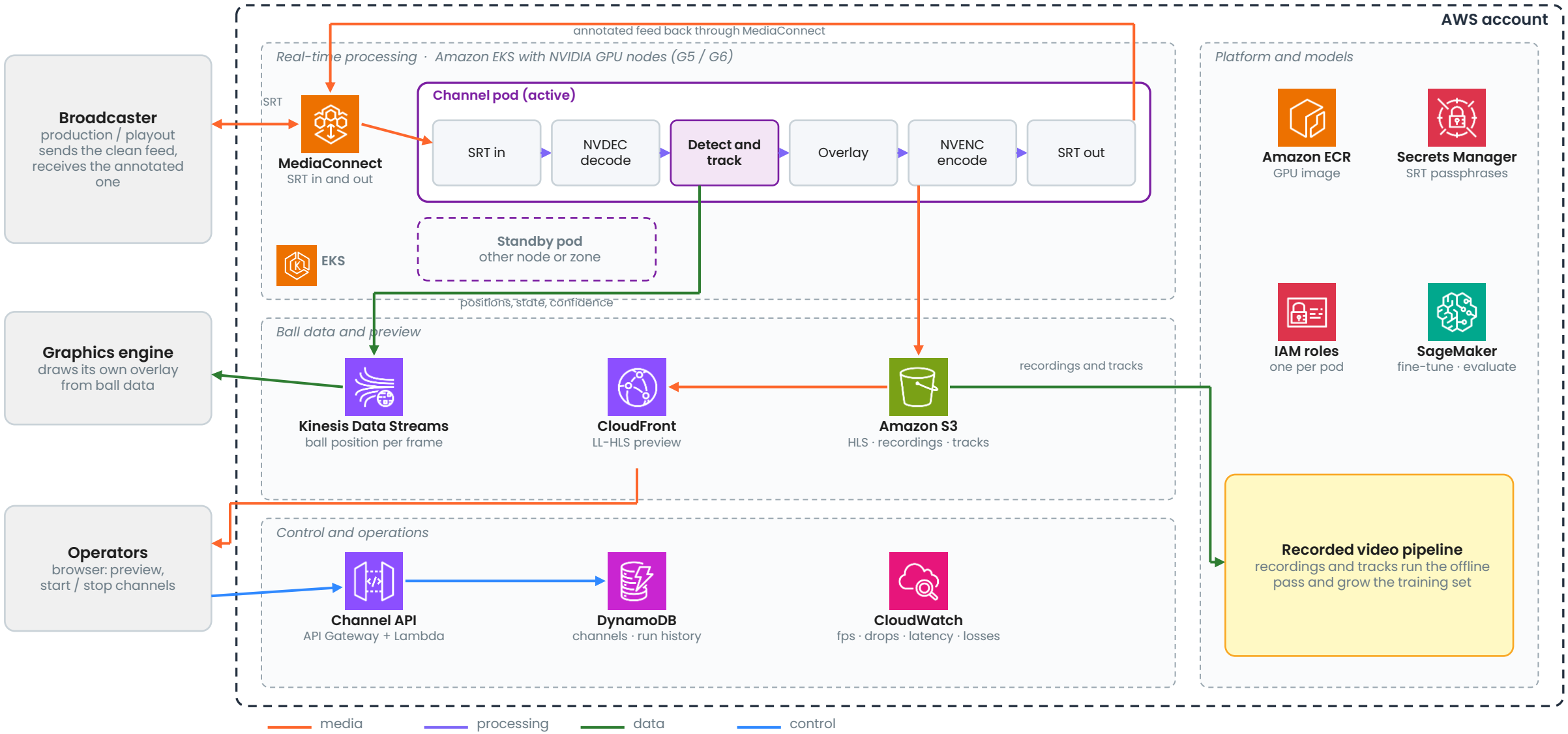
Architecture

Recorded Video: Technical Approach (2/2)

Aspect	Decision	Rationale
Media store	Amazon S3: raw and output buckets, lifecycle rules move raw media to cheaper tiers	Lowest cost per GB, and the event source for the pipeline
Trigger	S3 event to EventBridge; a rule submits the AWS Batch job	An upload starts the work; no polling, retries built in
Compute	AWS Batch with a GPU compute environment (G5 with A10G or G6 with L4), Spot where possible	Pay per job and scale to zero between batches; Fargate has no GPUs, so Batch manages the instances
Inference	YOLOE exported to TensorRT (or ONNX Runtime CUDA), FP16; NVDEC decode, NVENC encode	The prototype's runtime is laptop-specific; one NVIDIA GPU does decode, inference and encode
Two jobs	Job 1 detects and tracks and records the detections; Job 2 runs the offline pass and renders	The GPU-heavy pass runs once; the offline pass and rendering re-run cheaply with new settings
Metadata	Aurora PostgreSQL Serverless: jobs, shots and per-frame tracks	Queryable by shot and time; feeds analytics and the training set
Delivery	CloudFront in front of S3 (MP4 or HLS) with signed URLs	Edge delivery straight from S3, no media server to run
Model lifecycle	Amazon SageMaker: fine-tune, score on the hand-labelled set, register	Every model is measured on the same ground truth before it reaches the registry
Platform	Amazon ECR, Secrets Manager, IAM roles, CloudWatch	Images, secrets, keyless access between services, job metrics (fps, recall)

Live

Live Architecture on AWS (1/3)



Live

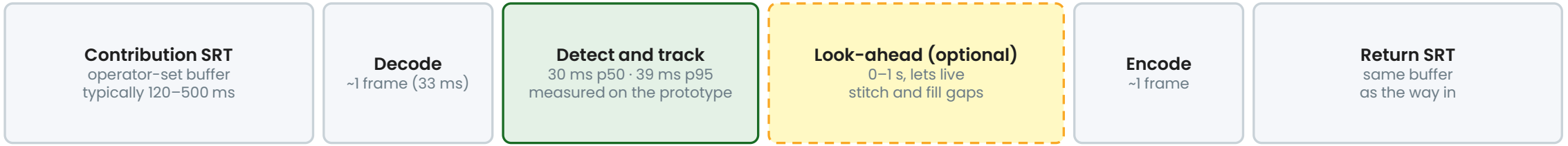
Live: Technical Approach (2/3)

Aspect	Decision	Rationale
Contribution in	AWS Elemental MediaConnect: SRT listener or caller, encrypted, delivered into our VPC	Managed broadcast transport: recovers lost packets, can fail over between two source feeds, no SRT servers to run
Compute	Amazon EKS with an NVIDIA GPU node group (G5 with A10G or G6 with L4); one pod per channel plus a standby pod	A live channel needs a dedicated GPU for the whole event; Fargate has no GPUs
Processing	The prototype's tracker: newest-frame reader, crop and full-frame detection, camera motion, two-hypothesis Kalman	One codebase for recorded and live video, already tested live-like on looped broadcasts
Decode and encode	NVDEC and NVENC on the same GPU, H.264 or HEVC	Keeps the CPU free; each adds about one frame of delay
Output	Annotated feed back to the broadcaster through MediaConnect; LL-HLS preview through S3 and CloudFront	The broadcaster keeps distribution; operators watch in a browser
Ball data feed	Per-frame ball position and state to Kinesis Data Streams; Firehose archives it to S3	Graphics engines can draw their own overlay; recorded tracks feed the offline pass and training
Control	Channel API on API Gateway and Lambda; channel settings and run history in DynamoDB	Start and stop channels and change settings without touching the cluster
Operations	CloudWatch with Container Insights: fps, dropped frames, latency, track losses, alarms	The same numbers the prototype measures, watched live

Live

Latency Budget and Live Considerations (3/3)

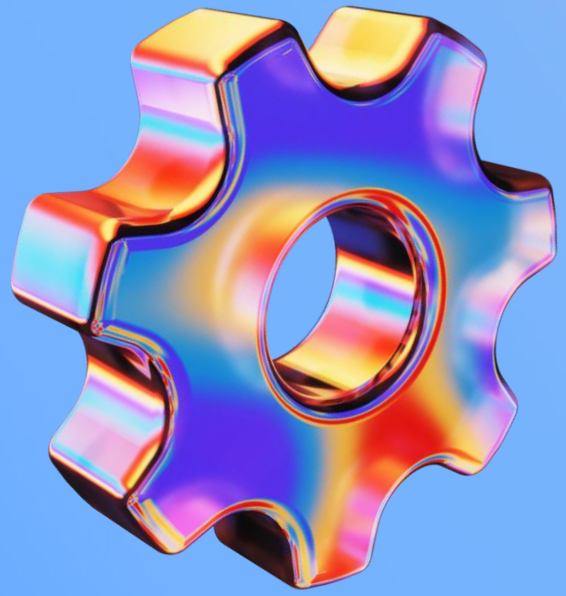
Where the delay comes from, per frame



Measured on the prototype (green); transport and coding are typical values to confirm on a real feed. Processing adds roughly 100 ms on top of the SRT buffers.

Considerations

- **Not measured yet:** speed on AWS GPU instances with TensorRT, and glass-to-glass latency on a real feed. Both are the first goals of a proof of concept.
- **50/60 fps input:** the per-frame budget halves to 17–20 ms. Crops would run every frame and full-frame passes spread across frames.
- **GPU capacity:** G5 / G6 instances are limited by per-region vCPU quotas, so request them early. Spot instances suit recorded video, not a live event.
- **Resilience:** MediaConnect can take two source feeds and fail over; a standby pod per channel; if processing fails, the clean feed should pass through unannotated rather than stop.
- **Burned-in or data:** sending the ball position as data lets the broadcaster's graphics own the look, and keeps our pipeline out of the picture path if they prefer.



Thanks!

southworks